Mid-Level AI Project Ideas

1.AI-Based Resume Screening System

Description : Build an AI system that automatically screens resumes and ranks candidates based on job requirements.

Steps:

- Collect resumes and job descriptions
- Extract text and preprocess using NLP techniques
- Generate embeddings using transformer models
- Match resumes to job descriptions using similarity scores
- Rank candidates and provide explainable insights

Step 1:

Collect resumes and job descriptions

Description:

Importing files: This gives Colab the ability to interact with your computer for uploads.

files.upload(): Opens a dialog so you can select resumes (PDF, DOCX, TXT). After uploading, they are stored in Colab's workspace.

uploaded.keys(): Collects the names of the files you uploaded.

Convert to list: Makes it easier to work with those filenames.

Print statement: Confirms which resumes were successfully uploaded.

Program:

```
from google.colab import files
# Upload resumes (PDF, DOCX, TXT)
```

```
uploaded = files.upload()
resume_files = list(uploaded.keys())
print("Uploaded resumes:", resume_files)
```

Output:

Choose Files 5 files

kam resume.pdf(application/pdf) - 337912 bytes, last modified: 7/7/2026 - 100% done
magi resume.pdf(application/pdf) - 329415 bytes, last modified: 7/7/2026 - 100% done
PDFGallery_20260707_142844.pdf(application/pdf) - 234625 bytes, last modified: 7/7/2026 - 100% done
srija resume.pdf(application/pdf) - 346382 bytes, last modified: 7/7/2026 - 100% done
vans resume.pdf(application/pdf) - 328374 bytes, last modified: 7/7/2026 - 100% done

```
Saving kam resume.pdf to kam resume (1).pdf
Saving magi resume.pdf to magi resume.pdf
Saving PDFGallery_20260707_142844.pdf to PDFGallery_20260707_142844.pdf
Saving srija resume.pdf to srija resume.pdf
Saving vans resume.pdf to vans resume.pdf
```

Step 2:

Extract text and preprocess using NLP techniques

Description:

Extract text: Reads resumes from PDF, DOCX, or TXT files

and converts them into plain text.

Preprocess text: Cleans the text using NLP (lowercase, remove

stopwords, lemmatize words).

Result: You get clean, meaningful text from each resume, ready for AI analysis.

Program 1:

```
!pip install PyPDF2 python-docx docx2txt


import PyPDF2, docx2txt


def extract_text(file_path):
text = ""
if file_path.endswith(".pdf"):
with open(file_path, "rb") as f:
reader = PyPDF2.PdfReader(f)
for page in reader.pages:
text += page.extract_text() + " "
elif file_path.endswith(".docx"):
text = docx2txt.process(file_path)
elif file_path.endswith(".txt"):
with open(file_path, "r", encoding="utf-8") as f:

text = f.read()
```

```
return text


resumes = [extract_text(f) for f in resume_files]
```

Output:

```
Collecting PyPDF2
  Downloading pypdf2-3.0.1-py3-none-any.whl.metadata (6.8 kB)
Collecting python-docx
  Downloading python_docx-1.2.0-py3-none-any.whl.metadata (2.0 kB)
Collecting docx2txt
  Downloading docx2txt-0.9-py3-none-any.whl.metadata (529 bytes)
Requirement already satisfied: lxml>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from python-docx) (6.1.1)
Requirement already satisfied: typing_extensions>=4.9.0 in /usr/local/lib/python3.12/dist-packages (from python-docx) (4.15.0)
Downloading pypdf2-3.0.1-py3-none-any.whl (232 kB)
   ---------------------------------------- 232.6/232.6 kB 2.1 MB/s eta 0:00:00
Downloading python_docx-1.2.0-py3-none-any.whl (252 kB)
   ---------------------------------------- 253.0/253.0 kB 18.1 MB/s eta 0:00:00
Downloading docx2txt-0.9-py3-none-any.whl (4.0 kB)
Installing collected packages: docx2txt, python-docx, PyPDF2
Successfully installed PyPDF2-3.0.1 docx2txt-0.9 python-docx-1.2.0
```

Program 2:

```
!pip install spacy

import spacy


nlp = spacy.load("en_core_web_sm")


def preprocess(text):

doc = nlp(text.lower())

tokens = [token.lemma_ for token in doc if not token.is_


stop and token.is_alpha]

return " ".join(tokens)
processed_resumes = [preprocess(resume) for resume in resumes]
```

Output:

```
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.14)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.13)
Requirement already satisfied: thinc<8.4.0,>=8.3.12 in /usr/local/lib/python3.12/dist-packages (from spacy) (8.3.13)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.5.3)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.0)
Requirement already satisfied: confection<2.0.0,>=1.3.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.3.3)
Requirement already satisfied: typer<1.0.0,>=0.5.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.25.1)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (4.67.3)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.2)
```

Step 3:

Generate embeddings using transformer models

Description:

Install sentence-transformers: This library provides pre-trained transformer models for text embeddings.

Load model: SentenceTransformer('all-MiniLM-L6-v2') loads a lightweight transformer that converts text into numerical vectors.

Generate embeddings:

resume_embeddings → turns each processed resume into a vector representation.

job_embedding → turns the job description into a vector.

Purpose: Embeddings capture the meaning of text so the AI can compare resumes and job descriptions mathematically.

Program:

```
!pip install sentence-transformers

from sentence_transformers import SentenceTransformer


model = SentenceTransformer('all-MiniLM-L6-v2')


resume_embeddings = model.encode(processed_resumes)

job_embedding = model.encode([processed_job])[0]
```

Output:

Step 4:

Match resumes to job descriptions using similarity scores

Description:

You write the job description as text (e.g., “Looking for a machine learning engineer with strong Python and NLP skills”).

Then you preprocess it using the same NLP cleaning method you applied to resumes (lowercase, remove stopwords, lemmatize).

This ensures the job description is in the same clean format as the resumes.

Purpose: By preprocessing both resumes and the job description, they can be compared fairly using embeddings and similarity scores.

Program:

```
job_description = "Looking for a machine learning engineer with strong
Python and NLP skills."
processed_job = preprocess(job_description)
```

Step 5:

Rank candidates and provide explainable insights

Description:

Cosine similarity: Measures how close each resume’s embedding is to the job description embedding (higher score = better match).

np.argsort(similarities)[::-1]: Sorts resumes from highest to lowest similarity score.

Ranking loop: Prints resumes in order, showing their rank, similarity score, and

filename.

Program:

```
from sklearn.metrics.pairwise import cosine_similarity

import numpy as np

similarities = cosine_similarity([job_embedding], resume_embeddings)[0]


ranked_indices = np.argsort(similarities)[::-1]

print("\nCandidate Ranking:")

for rank, idx in enumerate(ranked_indices, 1):

print(f"{rank}. Resume {idx+1} (Score: {similarities[idx]:.4f}) -
{resume_files[idx]}")
```

Output:

```
Candidate Ranking:
1. Resume 3 (Score: 0.1138) - resume (5) (1).pdf
2. Resume 2 (Score: 0.1138) - resume (6) (1).pdf
3. Resume 1 (Score: 0.1138) - resume (7) (1).pdf
4. Resume 4 (Score: 0.1138) - resume (4) (1).pdf
```

2. Intelligent Customer Support Chatbot

Description: Create an AI chatbot that understands user queries and provides accurate, context-aware responses.

Steps:

- Define intents and collect conversation data
- Preprocess text and train an NLP model
- Use transformer-based language models for responses
- Implement context handling and fallback logic
- Evaluate responses and improve accuracy

Step 1:

Define intents and collect conversation data

Description:

This code creates a small table of sample customer messages and their intents for chatbot training.

- data stores example text and labels.
- pd.DataFrame(...) turns it into a table with text and intent columns.
- df.head() shows the first few rows.

It is used as the starting dataset for building the chatbot.

Program:

```
import pandas as pd
import re
import numpy as np


data = [
     ("hi", "greeting"),
     ("hello", "greeting"),
     ("good morning", "greeting"),
     ("track my order", "track_order"),
     ("where is my package", "track_order"),
     ("my order has not arrived", "track_order"),
     ("i want a refund", "refund"),
     ("how do i get my money back", "refund"),
     ("cancel my order", "cancel_order"),
     ("i want to cancel my purchase", "cancel_order"),
     ("payment failed", "payment_issue"),
     ("my card was charged twice", "payment_issue"),
     ("change my address", "update_details"),
     ("update shipping address", "update_details"),
     ("speak to human agent", "human_handoff"),
     ("talk to support", "human_handoff"),
     ("bye", "goodbye"),
     ("thank you", "thanks"),
]


df = pd.DataFrame(data, columns=["text", "intent"])
```

```
df.head()
```

Output:

	text	intent
0	hi	greeting
1	hello	greeting
2	good morning	greeting
3	track my order	track_order
4	where is my package	track_order

Step 2:

Preprocess text and train an NLP model

Description:

This code preprocesses text and trains a simple

intent classification model for the chatbot.

- clean_text() converts text to lowercase and removes symbols and extra spaces.
- df["clean_text"] = ... creates a cleaned version of the message text.
- value_counts() checks how many examples each intent has.
- train_test_split() divides the data into training and test sets.
- TfidfVectorizer() converts text into numerical features.
- LogisticRegression() trains the classifier to predict the intent.
- accuracy_score() and classification_report() evaluate how well the model performed.

Program:

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"[^a-zA-Z0-9\s]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df["clean_text"] = df["text"].apply(clean_text)

print(df["intent"].value_counts())

X_train, X_test, y_train, y_test = train_test_split(
    df["clean_text"],
    df["intent"],
    test_size=0.25,
    random_state=42
)

vectorizer = TfidfVectorizer(ngram_range=(1, 2), max_features=5000)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

clf = LogisticRegression(max_iter=1000)
clf.fit(X_train_vec, y_train)
```

```python
pred = clf.predict(X_test_vec)

print("Accuracy:", accuracy_score(y_test, pred))

print(classification_report(y_test, pred))
```

Output:

```
•••  intent
     greeting          3
     track_order       3
     refund            2
     cancel_order      2
     payment_issue     2
     update_details    2
     human_handoff     2
     goodbye           1
     thanks            1
     Name: count, dtype: int64
     Accuracy: 0.0
                     precision    recall  f1-score   support

       cancel_order       0.00      0.00      0.00       1.0
           greeting       0.00      0.00      0.00       2.0
      payment_issue       0.00      0.00      0.00       0.0
        track_order       0.00      0.00      0.00       2.0
     update_details       0.00      0.00      0.00       0.0


           accuracy                           0.00       5.0
          macro avg       0.00      0.00      0.00       5.0
       weighted avg       0.00      0.00      0.00       5.0

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 i
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in l
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 i
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in l
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 i
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in l
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Step 3:

Use transformer-based language models for responses

Description:

This code sets up the chatbot's **response generation** part.

- pip install ... installs the required libraries for using Hugging Face models.
- AutoTokenizer and AutoModelForCausalLM load the pre-trained distilgpt2 model.
- pipeline("text-generation") creates a text generator that can produce natural-sounding replies.
- intent_responses stores fixed replies for each customer intent, like greeting, refund, or order tracking.

Program:

```
!pip -q install transformers sentencepiece accelerate


from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline


model_name = "distilgpt2"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(model_name)


generator = pipeline(

    "text-generation",

    model=model,

    tokenizer=tokenizer

)


intent_responses = {

    "greeting": "Hello! How can I help you today?",

    "track_order": "Please share your order ID so I can help track it.",

    "refund": "I can help with refunds. Please provide your order ID and

    Reason for refund.",
```

```
    "cancel_order": "I can help cancel your order. Please share your
    order  ID.",
    "payment_issue": "Sorry about the payment issue. Please tell me the
    error message or transaction ID.",
    "update_details": "I can help update your account or shipping details.
    Please confirm the changes needed.",
    "human_handoff": "I'm connecting you to a human support agent.",
    "goodbye": "Goodbye! Have a great day.",
    "thanks": "You're welcome!"
}
```

Output:

Step 4:

Implement context handling and fallback logic

Description:

This code adds **intent prediction, fallback handling,**

And conversation memory for the chatbot.

- conversation_history = [] stores each user message, predicted intent, confidence score, and bot reply.
- predict_intent(text) cleans the text, converts it into features, and predicts the most likely intent with a confidence score.
- generate_reply(user_text) checks the confidence:
    - if confidence is low, it gives a safe fallback message.
    - otherwise, it uses the language model to generate a reply based on the intent.
- ChatMemory saves recent conversation turns so the bot can remember short context.

- chat(user_text) uses that memory to make responses more context-aware.

Program:

```
conversation_history = []


def predict_intent(text):
    cleaned = clean_text(text)
    vec = vectorizer.transform([cleaned])
    probs = clf.predict_proba(vec)[0]
    classes = clf.classes_
    best_idx = np.argmax(probs)
    return classes[best_idx], float(probs[best_idx])


def generate_reply(user_text):
    intent, confidence = predict_intent(user_text)


    if confidence < 0.45:
        reply = "I’m not fully sure I understood. Could you rephrase or
    share  your order ID?"
    else:
        base_reply = intent_responses.get(intent, "I can help with that.")
        prompt = f"Customer support assistant. User said: {user_text}.
        Reply politely and clearly: {base_reply}"
        reply = generator(prompt)[0]["generated_text"].strip()


        conversation_history.append({
            "user": user_text,
            "intent": intent,
```

```
            "confidence": confidence,
            "bot": reply
        })
        return reply, intent, confidence

class ChatMemory:
    def __init__(self, max_turns=5):
        self.max_turns = max_turns
        self.turns = []

    def add(self, user, bot):
        self.turns.append((user, bot))
        self.turns = self.turns[-self.max_turns:]

    def get_context(self):
        return "\n".join([f"User: {u}\nBot: {b}" for u, b in self.turns])

memory = ChatMemory()

def chat(user_text):
    context = memory.get_context()
    intent, confidence = predict_intent(user_text)

    if confidence < 0.45:
        reply = "I’m not fully sure I understood. Could you rephrase or
    share your order ID?"
    else:
        base_reply = intent_responses.get(intent, "I can help with that.")
```

```
        prompt = f"{context}\nUser: {user_text}\nBot: {base_reply}"
        reply = generator(prompt)[0]["generated_text"].strip()

    memory.add(user_text, reply)
    return reply
```

Step 5:

Evaluate responses and improve accuracy

Description:

This code checks how well the intent model is working on sample customer messages.

- eval_samples contains example user texts and the correct intent for each one.
- predict_intent(text) predicts the intent and confidence for each message.
- The print() lines show:
  - the input text,
  - the predicted intent,
  - the confidence score,
  - and the expected intent.
- The line print("-" * 50) just separates each result for easy reading.

Program:

```
eval_samples = [
    ("track my package", "track_order"),
```

```
    ("cancel this order", "cancel_order"),
    ("my card got charged twice", "payment_issue"),
    ("hello", "greeting"),
    ("i want a refund", "refund"),
]


for text, expected in eval_samples:
    pred_intent, conf = predict_intent(text)
    print(f"Text: {text}")
    print(f"Predicted: {pred_intent} | Confidence: {conf:.2f} | Expected: {expected}")
    print("-" * 50)
```

Output:

```
Text: track my package
Predicted: payment_issue | Confidence: 0.15 | Expected: track_order
--------------------------------------------------
Text: cancel this order
Predicted: payment_issue | Confidence: 0.15 | Expected: cancel_order
--------------------------------------------------
Text: my card got charged twice
Predicted: payment_issue | Confidence: 0.25 | Expected: payment_issue
--------------------------------------------------
Text: hello
Predicted: payment_issue | Confidence: 0.16 | Expected: greeting
--------------------------------------------------
Text: i want a refund
Predicted: refund | Confidence: 0.29 | Expected: refund
--------------------------------------------------
```

3. AI-Powered Visual Defect Detection

Description: Develop a computer vision system that detects defects in manufactured products using images.

Steps:

- Collect and label product images
- Apply image preprocessing and augmentation
- Train a CNN or vision transformer model
- Evaluate performance using precision and recall
- Deploy the model for real-time inspection

Step 1:

Collect and label product images

Description:

Collect images of manufactured products using a camera or from an existing dataset. Organize the images into different folders based on their condition, such as Good and Defective, so the model can learn to distinguish between them.

Program:

```
from google.colab import drive

drive.mount('/content/drive')
```

```
import tensorflow as tf


DATA_DIR = '/content/drive/MyDrive/dataset/defect_dataset'


train_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset='training',
    seed=123,
    image_size=(224, 224),
    batch_size=32
)


val_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset='validation',
    seed=123,
    image_size=(224, 224),
    batch_size=32
)


class_names = train_ds.class_names
print(class_names)
```

Output:

```
Found 8648 files belonging to 2 classes.
Using 6919 files for training.
Found 8648 files belonging to 2 classes.
Using 1729 files for validation.
['casting_512x512', 'casting_data']
```

Step 2:

Apply image preprocessing and augmentation

Description:

Preprocess the images by resizing them to a fixed size, normalizing pixel values, and removing noise if necessary. Apply data augmentation techniques such as rotation, flipping, zooming, and brightness adjustment to increase the diversity of the dataset and improve the model's robustness.

Program:

```
from tensorflow.keras import layers


normalization = layers.Rescaling(1./255)


data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])
```

Step 3:

Train a CNN or vision transformer model

Description:

Train a deep learning model, such as a Convolutional Neural Network (CNN) or a Vision Transformer (ViT), using the preprocessed images. The model learns to identify patterns and features that differentiate defective products from non-defective ones.

Program:

```
model = tf.keras.Sequential([
    layers.Input(shape=(224, 224, 3)),
    normalization,
    data_augmentation,
    layers.Conv2D(32, 3, activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(128, 3, activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(len(class_names), activation='softmax')
])


model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)


model.fit(train_ds, validation_data=val_ds, epochs=10)
```

Output:

```
Epoch 1/10
217/217 ━━━━━━━━━━━━━━━━━━━━ 1505s 7s/step - accuracy: 0.8730 - loss: 0.3503 - val_accuracy: 0.9370 - val_loss: 0.1510
Epoch 2/10
217/217 ━━━━━━━━━━━━━━━━━━━━ 35s 159ms/step - accuracy: 0.9329 - loss: 0.1839 - val_accuracy: 0.7571 - val_loss: 0.6799
Epoch 3/10
217/217 ━━━━━━━━━━━━━━━━━━━━ 29s 136ms/step - accuracy: 0.9571 - loss: 0.1158 - val_accuracy: 0.4482 - val_loss: 1.9107
Epoch 4/10
217/217 ━━━━━━━━━━━━━━━━━━━━ 30s 138ms/step - accuracy: 0.9666 - loss: 0.0948 - val_accuracy: 0.8195 - val_loss: 0.6404
Epoch 5/10
217/217 ━━━━━━━━━━━━━━━━━━━━ 29s 134ms/step - accuracy: 0.9678 - loss: 0.0899 - val_accuracy: 0.3898 - val_loss: 3.3680
Epoch 6/10
217/217 ━━━━━━━━━━━━━━━━━━━━ 30s 136ms/step - accuracy: 0.9754 - loss: 0.0759 - val_accuracy: 0.3678 - val_loss: 3.2585
Epoch 7/10
217/217 ━━━━━━━━━━━━━━━━━━━━ 30s 139ms/step - accuracy: 0.9796 - loss: 0.0562 - val_accuracy: 0.3661 - val_loss: 3.6146
Epoch 8/10
217/217 ━━━━━━━━━━━━━━━━━━━━ 30s 138ms/step - accuracy: 0.9819 - loss: 0.0587 - val_accuracy: 0.2834 - val_loss: 3.0918
Epoch 9/10
217/217 ━━━━━━━━━━━━━━━━━━━━ 29s 136ms/step - accuracy: 0.9845 - loss: 0.0455 - val_accuracy: 0.5309 - val_loss: 2.1568
Epoch 10/10
217/217 ━━━━━━━━━━━━━━━━━━━━ 30s 140ms/step - accuracy: 0.9803 - loss: 0.0597 - val_accuracy: 0.5194 - val_loss: 2.2052
<keras.src.callbacks.history.History at 0x7804f19fb620>
```

Step 4:

Evaluate performance using precision and recall

Description:

Evaluate the trained model using the test dataset. Calculate Precision, which measures how many predicted defects are actually defective, and Recall, which measures how many actual defects are correctly detected. These metrics help determine the model's accuracy and reliability.

Program:

```
import numpy as np
from sklearn.metrics import precision_score, recall_score


y_true = []
y_pred = []


for images, labels in val_ds:
    preds = model.predict(images, verbose=0)
```

```
    y_true.extend(labels.numpy())
    y_pred.extend(np.argmax(preds, axis=1))

print("Precision:", precision_score(y_true, y_pred, average='weighted'))
print("Recall:", recall_score(y_true, y_pred, average='weighted'))
```

Output:

```
Precision: 0.8881649040564334
Recall: 0.5193753614806247
```

Step 5:

Deploy the model for real-time inspection

Description:

Deploy the trained model in a real-time inspection system by connecting it to a camera. As products pass in front of the camera, the model analyzes each image and classifies it as Good or Defective, enabling automatic quality inspection during manufacturing.

Program:

```
model.save('/content/defect_detection_model.keras')

loaded_model =
tf.keras.models.load_model('/content/defect_detection_model.keras')


def predict_image(img_path):
    img = tf.keras.utils.load_img(img_path, target_size=(224, 224))
    img = tf.keras.utils.img_to_array(img)
    img = tf.expand_dims(img, axis=0)
    pred = loaded_model.predict(img, verbose=0)
```

```
    return class_names[np.argmax(pred)], float(np.max(pred))
# Example:
# label, confidence = predict_image('/content/drive/MyDrive/test.jpg')
# print(label, confidence)
```